2장. 객체지향 프로그래밍

영화 예매 시스템으로 본 협력, 다형성, 합성

오브젝트 스터디 · Week 01
발표자: 이강욱

영화 예매 도메인

무엇을 예매하나

MOVIE
영화
제목 · 상영 시간 · 가격

1 : N
여러 번

SCREENING
상영
상영 일자 · 시간 · 순번

예매하는 대상은
영화가 아니라 상영이다
관객은 특정 시간의 상영을 보려고 돈을 낸다

요금은 어떻게 정하나

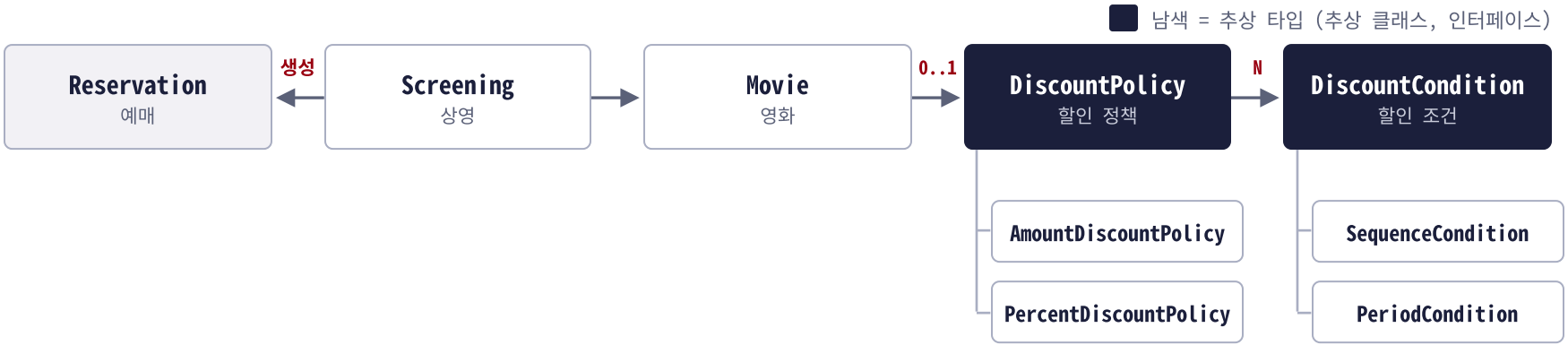
DISCOUNT CONDITION
할인 조건
할인 **여부**를 정한다
순서 조건: 상영 순번
기간 조건: 요일 · 시간대
정책마다 1개 이상
하나라도 만족하면 할인

1..N : 1
1개 이상

DISCOUNT POLICY
할인 정책
할인 **금액**을 정한다
금액 할인: 800원 빼기
비율 할인: 10% 빼기
영화당 0~1개

FEE
예매 요금
(정가 - 할인) × 인원수
아바타 · 월 10:30 · 2명
(10,000 - 800) × 2
= 18,400원

도메인 구조를 따르는 클래스



DOMAIN

도메인을 따르는 구조

클래스 이름을 도메인 개념 이름과 맞춘다
→ 코드만 보고도 도메인이 읽힌다

AUTONOMY

자율적인 객체

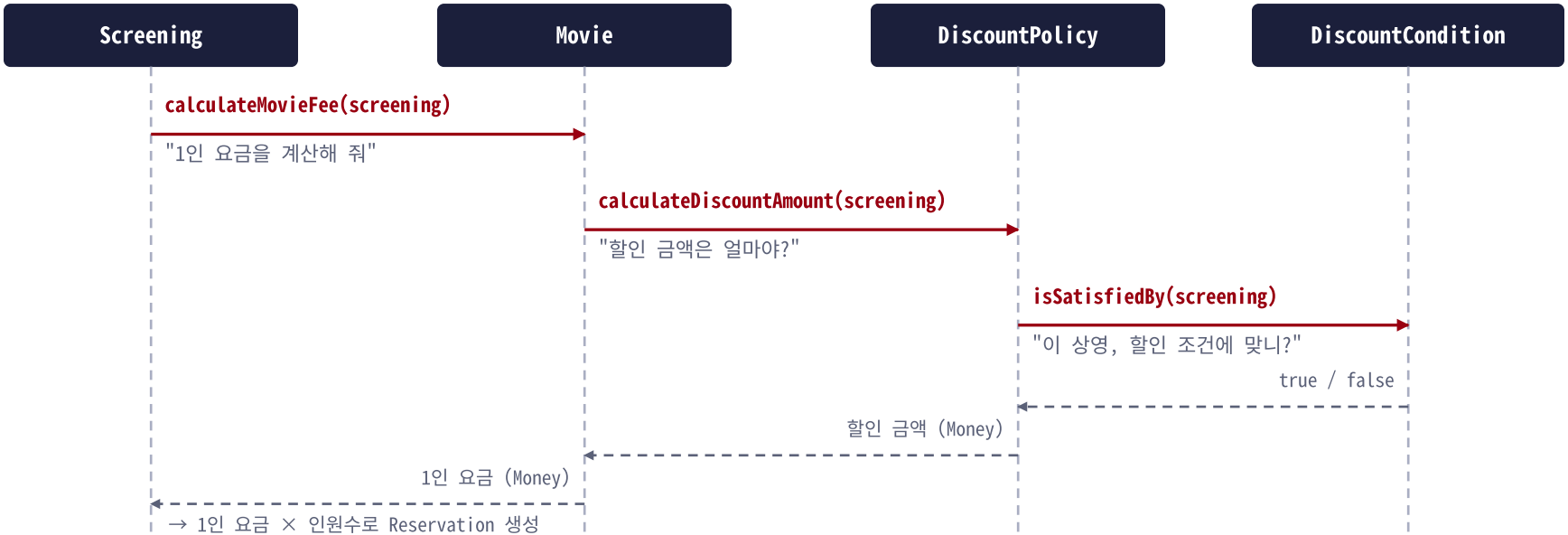
상태는 private, 행동만 public
→ 외부 간섭 없이 스스로 판단한다
→ 내부 구현을 자유롭게 바꾼다

VALUE OBJECT

의미 있는 값은 객체로

Long 대신 Money
→ "금액"이라는 의미가 드러난다
→ plus · minus · times를 한곳에

객체는 메시지로 협력한다



메서드 호출(X), 메시지 전송(O) Screening은 Movie에 그 메서드가 있는지 모른다. 응답할 수 있다고 믿고 보낼 뿐이다.

할인 정책: 템플릿 메서드 패턴

step01 / DiscountPolicy.kt

```
abstract class DiscountPolicy(vararg conditions: DiscountCondition) {
    private val conditions = conditions.toList()

    // 템플릿 메서드: 흐름은 부모가 정한다
    fun calculateDiscountAmount(screening: Screening): Money {
        for (condition in conditions) {
            if (condition.isSatisfiedBy(screening)) {
                return getDiscountAmount(screening)
            }
        }
        return Money.ZERO
    }

    // 추상 메서드: 자식이 구현한다
    protected abstract fun getDiscountAmount(screening: Screening): Money
}
```

1 **템플릿 메서드** (변하지 않는 흐름)

calculateDiscountAmount

공통 흐름은 부모에. 기본이 final이라 자식이 바꿀 수 없다

2 **추상 메서드** (달라지는 부분)

getDiscountAmount

abstract라 자식은 반드시 구현, protected라 밖에서는 못 부른다

자식 클래스의 구현

AmountDiscountPolicy → 고정 금액 (800원)
PercentDiscountPolicy → 정가 × 비율 (10%)

새 할인 정책은 **getDiscountAmount 하나만** 구현하면 된다. 어떤 구현이 실행될지는 실행 시점에 정해진다(오버라이딩).

코드의 의존성 ≠ 실행의 의존성

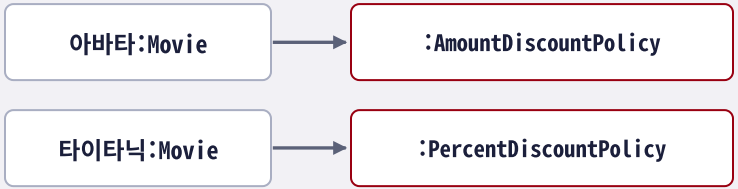
코드 · 클래스 사이



Movie는 추상 타입만 안다
금액/비율을 고르는 if가 코드에 없다

≠

실행 · 객체 사이



연결은 객체를 만드는 쪽(Main.kt)이 정한다

UPCASTING

업캐스팅

자식이 부모 타입 자리를 대신한다

POLYMORPHISM

다형성

같은 메시지, 객체마다 다른 응답

LATE BINDING

지연 바인딩

실행할 메서드를 실행 시점에 정한다

의존성이 다를수록 **유연해지지만**, 코드만 읽고 동작을 알기는 **어려워진다**. 유연성과 가독성의 트레이드오프

지연 바인딩 vs 초기 바인딩

LATE BINDING · 동적 바인딩

실행 시점에 연결한다

변수가 가리키는 **실제 객체**를 보고 고른다

step04 / Movie.kt

```
fee.minus(discountPolicy
    .calculateDiscountAmount(screening))
```

javap -c Movie.class

```
invokeinterface DiscountPolicy
    .calculateDiscountAmount
```

AmountDiscountPolicy라는 이름은 바이트코드에 없다.
실행할 때 실제 객체의 클래스에서 구현을 찾는다

EARLY BINDING · 정적 바인딩

컴파일 시점에 연결한다

변수에 **선언된 타입**을 보고 고른다

컴파일된 한 줄 → 실행되는 메서드는 넷

그 순간 들어 있는 객체	실행되는 메서드
AmountDiscountPolicy	**금액 할인** (아바타)
PercentDiscountPolicy	**비율 할인** (타이타닉)
NoneDiscountPolicy	**할인 없음** (스타워즈)
Percent… (정책 교체 후) 같은 아바타 객체	**비율 할인**

초기 바인딩이었다면 이 한 줄은 **메서드 하나에 고정**됐다. 지연 바인딩이라서 **다형성**이 동작한다

같은 객체, 다른 바인딩

같은 AmountDiscountPolicy 객체에 세 가지 호출

```
val policy: DiscountPolicy = AmountDiscountPolicy(...)

// ① 오버라이딩
policy.calculateDiscountAmount(screening) // Amount 구현

// ② 오버로딩
fun describe(p: DiscountPolicy) = "할인 정책"
fun describe(p: AmountDiscountPolicy) = "금액 할인 정책"
describe(policy)   // "할인 정책"

// ③ 확장 함수
fun DiscountPolicy.name() = "DiscountPolicy"
fun AmountDiscountPolicy.name() = "AmountDiscountPolicy"
policy.name()      // "DiscountPolicy"
```

1 **오버라이딩 → 지연**
실제 객체를 따라간다 · Amount의 구현 실행

2 **오버로딩 → 초기**
인자의 선언 타입으로 고른다 · "할인 정책"

3 **확장 함수 → 초기**
멤버처럼 보여도 정적으로 결정 · "DiscountPolicy"

Q. Main.kt에 AmountDiscountPolicy라고 적었으니 초기 바인딩 아닐까?

Main.kt는 **어떤 객체를 넣을지**(의존성 주입)를 정하고, 바인딩은 **메서드를 호출하는 줄**(Movie.kt)에서 일어난다

Main.kt를 읽는 사람은 알지만 Movie는 모른다 → 정책을 설정·입력으로 골라도 Movie는 다시 컴파일할 필요가 없다

코드는 네 단계로 바뀐다

추상화에 의존하면, 기존 코드를 고치지 않고 확장할 수 있다

step 01

null 분기

할인 없음을 Movie가 null로 처리한다

개선 전

step 02

NoneDiscountPolicy

없음도 하나의 할인 정책으로 본다

Null Object

step 03

DefaultDiscountPolicy

인터페이스를 분리해 쓰지 않는 구현을 물려받지 않는다

트레이드오프

step 04

changeDiscountPolicy()

합성이라 실행 중에 정책을 바꿀 수 있다

합성

실행 결과 · 네 단계 모두 같다 (구조만 바뀌고 동작은 그대로)

아바타 월 10:30 · 3회차 · 2명
18,400원

아바타 수 19:00 · 5회차 · 2명
20,000원

타이타닉 화 15:00 · 3회차 · 1명
9,900원

스타워즈 수 19:00 · 1회차 · 3명
30,000원

step02: 없음도 하나의 정책

step01 / Movie.kt

```
fun calculateMovieFee(screening: Screening): Money {
    if (discountPolicy == null) {
        return fee
    }
    return fee.minus(
        discountPolicy.calculateDiscountAmount(screening))
}
```



step02 / NoneDiscountPolicy.kt

```
class NoneDiscountPolicy : DiscountPolicy() {
    override fun getDiscountAmount(
        screening: Screening): Money = Money.ZERO
}

// Movie: 정책이 있든 없든 똑같이 묻는다
fee.minus(discountPolicy.calculateDiscountAmount(screening))
```

X

정책이 없으면 Movie가 DiscountPolicy에 묻지 않고 직접 처리한다

할인 금액을 정하는 책임이 두 곳에 흩어진다

O

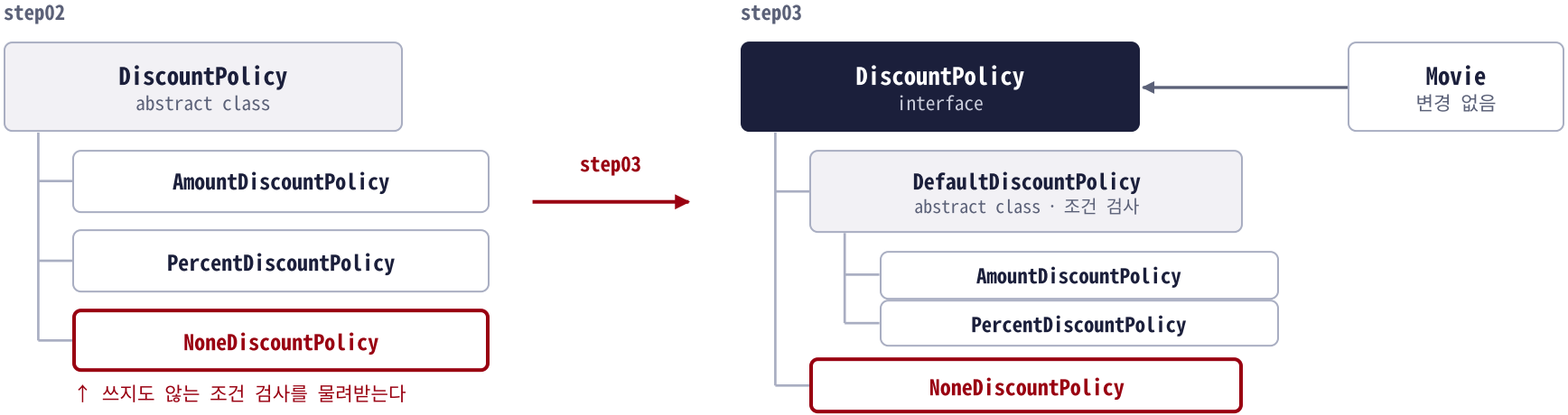
할인 금액은 언제나 DiscountPolicy가

Movie의 예외 분기가 사라진다

널 객체 패턴(Null Object) "없음"을 아무 일도 하지 않는 객체로 표현한다

기존 클래스를 고치지 않고 클래스 하나를 추가해 확장했다. 유연성이 필요한 곳에 추상화를 사용하라.

step03: 추상 클래스를 인터페이스로



PROBLEM

호출되지 않는 메서드

None의 getDiscountAmount는 한 번도 호출되지 않는다. 부모의 내부 사정과 개념적으로 결합된다

RESULT

Movie는 그대로

Movie는 처음부터 DiscountPolicy 인터페이스에만 의존했다. 한 줄도 고치지 않았다

KOTLIN

final override

override 멤버는 기본이 open이다. final로 막아 템플릿 흐름을 지킨다

상속보다 합성 (step04)

INHERITANCE

상속 · 렌즈 일체형 카메라

```
class AmountDiscountMovie : Movie()
```

· 부모가 내부에서 무엇을 부르는지 알아야 한다 → 캡슐화 약화
· 할인 방식이 클래스 선언에 고정된다 → 바꾸려면 객체를 새로 만든다

COMPOSITION

합성 · 렌즈 교환식 카메라

```
avatar.changeDiscountPolicy(
    PercentDiscountPolicy(0.1, ...))
```

· 메시지로만 협력한다 → 내부 구현을 몰라도 된다
· 몸체(Movie)는 그대로, 부품(정책)만 바꾼다

step04 실행 결과

같은 아바타 객체 · 월 10:30 · 3회차 · 2명

18,400원
금액 할인 800원

→

18,000원
비율 할인 10%

객체를 새로 만들지 않고 부품만 갈아 끼웠다

TOGETHER

상속과 합성은 함께 쓴다

Movie → DiscountPolicy: **합성**
DiscountPolicy 계층: **인터페이스 상속**

코드 재사용은 합성, 다형성은 인터페이스 상속

"실행 시점에 바꾼다"는 무슨 뜻일까

Q. "아바타를 비율 할인으로 바꿔 주세요"

Main.kt 한 줄 고치면 되지 않나?

```
PercentDiscountMovie("아바타", ...)        // 상속
Movie("아바타", ..., PercentDiscountPolicy()) // 합성
```

맞다. 둘 다 생성 코드 한 줄이면 된다.
하지만 이건 **실행 전에 소스를 고친 것**이다

켜져 있는 서버에서 avatar.changeDiscountPolicy()



Screening들은 같은 avatar를 **가리키기만** 한다.
필드 하나를 바꾸면 모두 다음 예매부터 새 정책을 쓴다

	개발 시점 변경	실행 시점 변경
누가	개발자	사용자·관리자·설정
방법	수정→빌드→배포	실행 중 부품 교체
상속	생성 코드만 수정	**새 객체 + 참조 교체**
합성	생성 코드만 수정	**부품 교체 한 번**

헷갈리기 쉬운 점

상속 버전도 지연 바인딩이다

getDiscountAmount 호출은 실행 시점에 실제 객체로 연결된다. 차이는 **"어떤 구현을 쓸지"가 어디에 있느냐**다

- 상속: 객체의 **클래스**에 (생성 후 고정)
- 합성: 객체의 **필드**에 (다시 대입 가능)

반박 ①: NoneDiscountPolicy는 과할까

과하지 않은 이유

- 0 **빈자리 하나를 채울 뿐**, 새 추상화가 아니다
- 0 **지금 있는 요구사항**, 할인 없는 영화
- 0 **분기 위치를 옮길 뿐**, Movie 분기가 사라진다

그래도 드는 의문

- ? null 처리가 Movie 한 곳뿐이다
- ? Movie만 읽으면 "할인 없음"이 안 보인다

코틀린이라면 한 줄로도 된다

```
fee.minus(discountPolicy?.calculateDiscountAmount(screening)
        ?: Money.ZERO)
```

하지만 "0원" 규칙은 여전히 Movie가 안다

내 결론

과한 설계는 아니다.
꼭 해야 하는 설계도 아니다.

책이 문제 삼는 건 NPE가 아니라 책임의 위치

그럼 "없음"은 언제 객체로 만들까?

상황	선택
"없음"이 정상 도메인 개념 (할인 0원)	**널 객체**
"없음"이 오류나 누락을 뜻함	**null 명시 처리, 예외**
null 처리가 한두 곳뿐	**코틀린 ?:**
쓰는 곳이 많은 타입, 라이브러리	**널 객체 (NOPLogger)**

반박 ②: changeDiscountPolicy는 필요할까

step04 / Movie.kt

```
class Movie(
    private val title: String,
    ...
    private var discountPolicy: DiscountPolicy
) {
    fun changeDiscountPolicy(discountPolicy: DiscountPolicy) {
        this.discountPolicy = discountPolicy
    }
}
```

1 **사실상 setter다**
누가, 언제, 왜 바꾸는지가 코드에 없다

2 **영향 범위가 안 보인다**
같은 Movie를 쓰는 모든 Screening의 요금이 한순간에 바뀐다

3 **불변성과 부딪힌다**
val이던 필드를 var로 바꿔야 한다

"실행 시점에 바꿀 수 있다" = 설계가 **허용한다**는 뜻. 교체 메서드를 꼭 열어 둘 필요는 없다

바꿀 요구사항이 없다
val 유지, 교체 메서드 없음 (step03 구조)

정책 변경이 실제 요구사항
withDiscountPolicy()로 새 객체, 또는 "기간별 할인 정책" 같은 도메인 개념

개념을 보여 주는 예제
지금처럼 두되 예시용임을 밝힌다

실무: 할인 이벤트는 데이터로 연다

1 · 운영자
백오피스에서 등록

```
PERCENT · 0.2
10:00 ~ 23:59 · ALL
```

→

2 · 서버 (요청마다)
DB 조회 → 정책 조립

```
factory.create(promo)
→ PercentDiscountPolicy
```

→

3 · PeriodCondition
시간 판단은 코드가

```
09:59 요청 → 할인 없음
10:00 요청 → 20% 할인
```

데이터로 끝나는 변경 vs 코드가 필요한 변경

요구사항	처리
할인율 20% → 30%	**데이터 수정**
기간 · 대상 상품 · 금액↔비율	**데이터 수정**
처음 보는 종류의 할인 1+1 · 세 번째 무료 · 첫 구매 한정	**새 클래스 → 배포**

새 종류도 클래스 하나 + 팩토리 한 줄. 가격 계산 코드는 그대로(OCP)

실무의 "실행 중에 바뀐다"

DB 데이터가 바뀌면
다음 요청부터 다른
정책 객체가 조립된다

정책이 클래스에 박혀 있지 않고
끼울 수 있는 부품이어야 한다
→ 합성이 전제

할인이 1년에 두어 번이라면
코드 수정·배포가 더 합리적일 수 있다

반박 ③: 인터페이스 분리는 과할까

내 결론

기준은 프로젝트 규모가 아니라
지금 갈라지는 지점이 있느냐

영화 예제는 구현 셋이 지금 동시에 쓰인다 → 과하지 않다

이건 오버엔지니어링

```
interface UserService { ... }
class UserServiceImpl : UserService
    // 구현은 영원히 이거 하나
```

지연 바인딩의 비용(추적이 어려움)만 내고
유연성은 얻지 못한다 · **YAGNI**

상황	선택
구현이 하나뿐	**구체 클래스로 시작** **둘째가 생기면 추출**
외부 시스템 경계 결제 · 메일 · 외부 HTTP	**하나여도 분리**
시간 · 난수 Clock · Random	**하나여도 분리**
도메인 → 인프라 의존	**분리 (의존성 역전)**
메서드 하나만 갈라짐	**함수 타입** **(Screening)->Money**
종류가 닫혀 있고 연산이 자주 늘어남	**sealed class + when**

추상화는 **가독성을 내고 유연성을 사는 거래**다. 변경이 눈앞에 있으면 분리하고, 상상 속에만 있으면 미룬다

유연한 설계는 추상화에서 시작된다

정리

1. **객체는 메시지로 협력한다**
   메서드 호출이 아니라 메시지 전송
2. **추상화에 의존하면 확장이 쉽다**
   다형성 · NoneDiscountPolicy
3. **재사용은 상속보다 합성**
   캡슐화 + 실행 중 교체

토론거리

Q1. NoneDiscountPolicy, DefaultDiscountPolicy를 적용하시겠어요?

널 객체 vs ?: Money.ZERO
인터페이스 vs 추상 클래스

Q2. 영화의 할인 정책을 실제로 바꿔야 한다면 어떻게 구현할까요?

changeDiscountPolicy(setter) vs 새 객체 반환 vs 기간별 정책